Pagesmith — The Skeleton & Catalogue

A Developer Guide to the Engine

David Falk

Abstract

Pagesmith is a generic, JSON-driven page-builder engine written in plain, dependency-free JavaScript. This guide explains how that engine works end to end, for a developer who wants to build on or extend it. We cover the philosophy (longevity, the engine-versus-instance split), the data model (a page is a flat array of typed blocks in a JSON file), the boot and load order, the event-driven render data-flow, and — the centrepiece — *the catalogue and the part contract*: how to write a self-contained component and the rules it must obey so it composes with every other part and stays portable across sites. We finish with the editing protocol, the two namespaces, theming, the shape-generic backend, and a checklist for standing up a new site. The deep internals of the inline editor are a separate guide; this one documents the producer side a part author needs.

Contents

1 Philosophy: why it is built this way

Before any code, it is worth understanding *why* Pagesmith looks the way it does, because every decision downstream falls out of three commitments.

1.1 Plain, dependency-free JavaScript

There is no framework here. No React, no Vue, no build step, no `npm install` on the front end, no bundler, no transpiler. Every file under `app/javascript/` is an IIFE — an old-fashioned `(function () { ... })()` — that runs directly in the browser exactly as written. The catalogue builds DOM with `document.createElement`. State lives in a JSON file. Events are plain `CustomEvent`s.

This is a deliberate, almost stubborn choice, and the reason is **longevity**. A framework is a bet that a particular toolchain will still be installable, patchable, and well-understood in five or ten years. That bet frequently loses: the build breaks, a transitive dependency is abandoned, a major version forces a rewrite. Plain JavaScript served as static files has no such failure mode. If a browser can run it today, it will run it in a decade. The cost is that you write a little more by hand; the payoff is that the thing keeps working with zero maintenance. For a tool whose whole job is to outlive the attention span of whoever set it up, that trade is the entire point.

> *Aside.* You will notice the code is written in a conservative dialect: `var` not `let`, `function` expressions in the hot paths, `[].forEach.call(...)` for NodeLists. That is not nostalgia — it is the same longevity instinct. The fewer language features you lean on, the fewer ways the floor can move under you.

1.2 The engine-versus-instance split

Pagesmith is two halves that never blur into each other (Figure 1). The **engine** knows *nothing* about your site — not its name, colours, pages, or social links. It reads all of that from the **instance** at runtime. The contract is strict and one-directional: the instance depends on the engine; the engine never depends on the instance. The single bridge is `site-config.js`, a plain manifest of values, plus the optional `site-blocks.js`, where you register parts that only make sense for one site.

The reason this matters is **reuse without forking**. Because the engine carries no site-specific strings, the same battle-tested files drop into any number of sites unchanged. Bug fixes and new parts flow to every site by copying engine files; nothing site-specific ever has to be untangled first. To stand up a new site you change values, not code.

> *The litmus test for "does this belong in the engine?"* If a line of code mentions a brand, a colour by name, a specific page, or a specific URL, it is *instance*, and it belongs in `site-config.js` / `site-blocks.js` / `data/`. The engine deals only in *roles* and *shapes*.

2 The data model: a page is a JSON file

This is the keystone, so go slowly. **A page is a JSON file.** That is the whole model. A page lives at `data/page-<slug>.json` and has this shape:

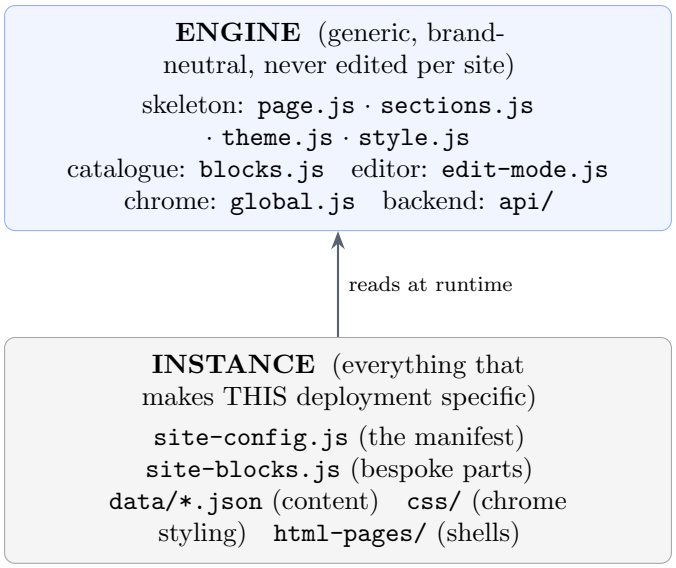

Figure 1: The one-directional dependency. The instance reads the engine; the engine never reads the instance.

```
{
  "title": "Showcase",
  "theme": { "accent": "#ff3b8e" },
  "sections": [
    { "id": "sc-h",     "type": "heading", "visible": true, "text": "Component showcase"
    },
    { "id": "sc-intro", "type": "text",    "title": "The catalogue",
      "body": "These are the building blocks Pagesmith ships with." }
  ]
}
```

Three top-level keys, and only `sections` is required: `title` (optional, sets the tab title), `theme` (optional, a per-page colour override; absent → inherit the site default from `global.json`), and `sections` — an **ordered, flat array of typed blocks**, which *is* the page.

2.1 Blocks are flat and typed

Every entry in `sections[]` is a **block**: a plain object whose `type` names a part in the catalogue, plus whatever data that part needs. The engine guarantees each block an *envelope* of fields, filling them in if absent (`normalize()` in `blocks.js`):

Field	Meaning
`id`	stable identifier; the editor keys reorder/hide/remove on it
`type`	which catalogue part renders it; unknown → falls back to `text`
`visible`	`false` hides the block on the published page
`style`	optional per-block inline styling (see `style.js`)

Everything *else* on the block is part-specific payload: a `gallery` carries `images:[...]`; a `chart` carries `kind` and `series:[...]`; a `spotlight` carries `value/label/caption`. The engine does not care — it hands the whole block to the part and lets the part read what it needs.

> *Why flat, not a tree?* A flat array is trivial to reason about, reorder (just array-index manipulation), diff, and store. Nesting *is* supported — the `group` part holds a `children:[...]` array

and renders it recursively — but nesting is opt-in, owned by one part, rather than baked into the model. The model stays a list; complexity lives in the parts that want it.

The shared site chrome (header, footer, nav, social, theme default) lives in one more file of the same family, `data/global.json`. It is not a page (no `sections[]`) but flows through the very same machinery (see `global.js` and Section 4).

3 Boot and load order

A page is an almost-empty HTML shell (`html-pages/page.html`): a `<main></main>` and little else, with *no* per-page CSS or JS. The body is built entirely from `sections[]` at runtime. So: what scripts load, in what order, and why does order matter?

The `<head>` loads two scripts synchronously: `site-config.js` then `media-base.js`. **`site-config.js` is first, always.** It defines `window.SITE_CONFIG` and the read-only accessor `window.SITE`. Being synchronous and first, `window.SITE` is guaranteed to exist before any other engine code runs — which matters because the very next script, `media-base.js`, already asks `SITE.isProd(host)` to decide whether to load the editor and which storage container to read.

`media-base.js` is the bootstrapper. It patches `window.fetch` (redirecting `/data/X.json` to the right blob container, and rewriting `/media-content/...` references to real storage URLs everywhere — JSON bodies, DOM attributes, CSS rules — via a `MutationObserver`), and then *injects the rest of the engine in a deliberate order* (Figure 2).

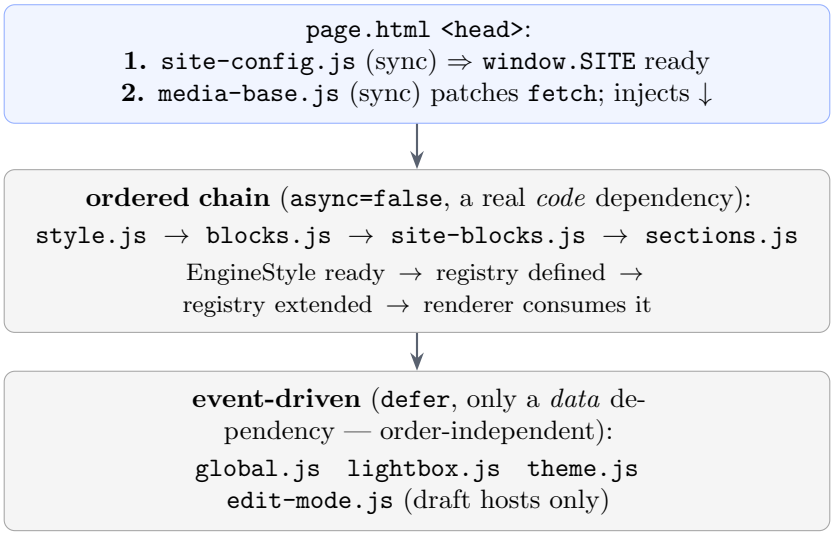

Figure 2: Load order. The catalogue chain uses ordered execution; theming and chrome use events.

Two mechanisms are at work, doing different jobs:

1. **Ordered execution** (`async=false`) for `style.js` → `blocks.js` → `site-blocks.js` → `sections.js`. Injected scripts default to `async` ("run whenever downloaded" — a race); setting `async=false` forces execution in insertion order regardless of download timing. This yields a strict dependency chain: `style.js` publishes `EngineStyle` (the catalogue calls `EngineStyle.apply()` on render); `blocks.js` defines the registry `window.Catalog`; `site-blocks.js` runs after it and adds to it; `sections.js` runs last, so the full registry (generic + bespoke) exists before it renders.

2. **Event-driven** (`defer`) for everything else. `global.js` and `theme.js` do not care *when* they load relative to `page.js`, because they listen for `engine-data-ready` (with a fallback for the case where the event already fired).

> *Why two mechanisms?* The catalogue chain has a real *code* dependency (`sections.js` literally calls `Catalog.render`), so it needs hard ordering. Theming and chrome only have a *data* dependency, so they use events and become order-independent — more robust than ordering everything.

4 The render data-flow

Now the heart: how data becomes DOM. The flow is **event-driven and order-independent** (Figure 3).

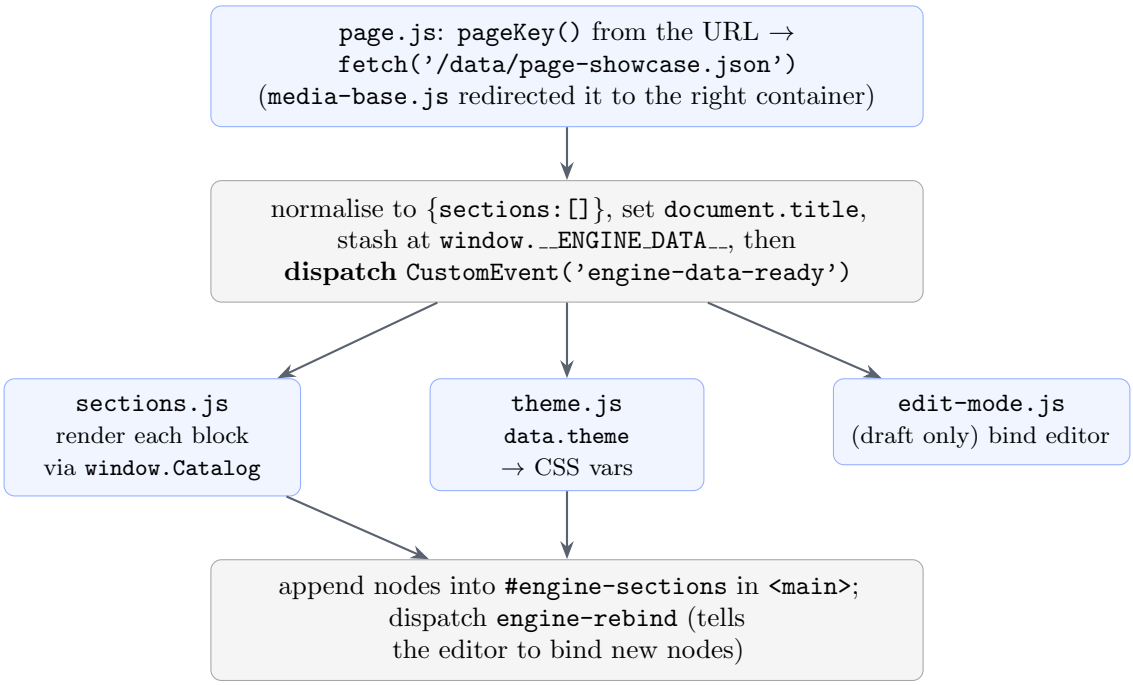

Figure 3: The render data-flow. `page.js` *announces*; `sections.js`, `theme.js`, and the editor each *react*.

Concretely: `page.js` runs on `DOMContentLoaded`. `pageKey()` derives a content-file key from the URL in priority order — an explicit `<meta name="engine-page">`, then a `/p/<slug>` route, then a clean top-level slug, defaulting to `home` — sanitising the slug to `[a-z0-9-]` so the filename can never traverse out of `/data/`. It then fetches `/data/<key>.json`. A missing file is *not* an error: a brand-new page has no JSON yet, so a 404 becomes an empty page the editor can fill. `page.js` normalises the data, sets the title, stashes it on `window.__ENGINE_DATA__`, and fires `engine-data-ready`.

That event is the linchpin. `page.js` does not *call* the renderer or themer; it *announces* the data and lets anyone interested react. Each listener also checks `window.__ENGINE_DATA__` on startup, to catch the case where the event fired *before* the listener attached. This is the order-independence in action: the load order can shuffle and the page still composes.

`sections.js` is intentionally ignorant — it never hard-codes a block type. For each entry it builds a small `ctx` and calls `window.Catalog.render(block, ctx)`, appending the node

into `#engine-sections`. New part types appear with *zero* changes here. When done it fires `engine-rebind` so the editor can bind the new nodes.

> *Note on `global.json`.* The shared chrome flows through the *same* event: `global.js` fetches it, hydrates header/footer, and dispatches `engine-data-ready` with `file:'global'`. `sections.js` ignores that one (global is chrome, not a page); `theme.js` applies its theme as the site-wide default before a page's own theme overrides it.

5 The catalogue and the part contract

This is the centrepiece. The catalogue (`blocks.js`) is a **registry of part types**. Each entry is a fully self-contained component bundling four things that a framework would scatter across files: its **markup** (a `render()` returning a DOM node), its **styling** (CSS injected once, namespaced `catalog-`), its **edit schema** (`fields[]`/`lists{}` the editor reads), and its **behaviour** (an optional `init()`). Adding a capability to the *entire system* — a section anyone can add, that edits, themes, and persists — is adding one object to this registry. That containment is what makes the engine portable.

5.1 Anatomy of a part

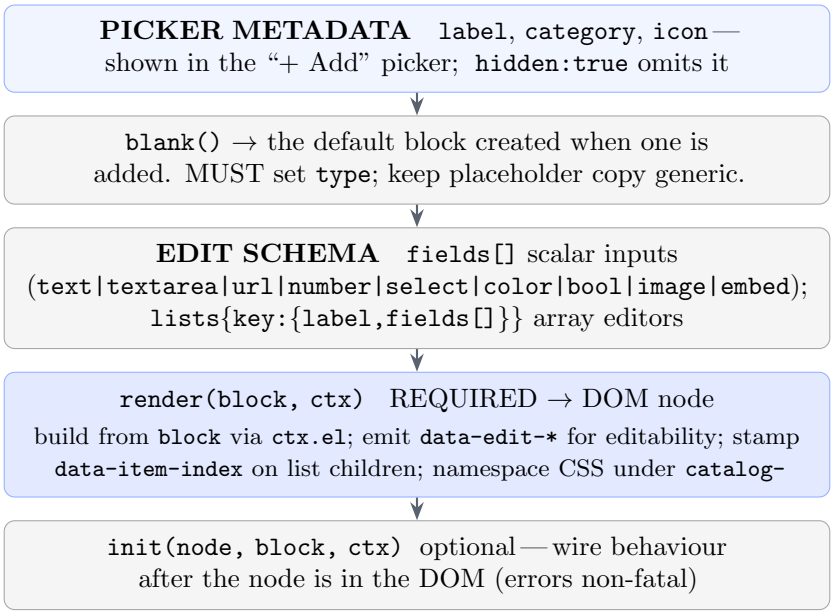

Figure 4: Anatomy of a part: every key a registry entry may define.

5.2 The `ctx` object

Every `render(block, ctx)` (and `init`) receives a small context built by the renderer. It is the part's entire window onto the engine:

ctx key	What it is / why you need it
ctx.el	tiny DOM builder el(tag, attrs, kids); text/html are special-cased, all else becomes an attribute
ctx.mediaUrl	resolves a /media-content/... path to the real (blob/local) URL so your <img src> works everywhere
ctx.file	the content-file key (e.g. page-showcase); goes into data-edit-file
ctx.index	this block's index in sections[]; handy for unique ids (e.g. a chart gradient) and lightbox grouping
ctx.path	the data-path to this block, e.g. sections[2] — **the base for every data-edit-* path you emit**

The block argument is just the block's data — the same object that sat in the JSON sections[].

5.3 Two real parts, read closely

The smallest useful part — text:

```
text: {
  label: 'Text', category: 'Basic', icon: '|P|',
  blank: function () { return { type: 'text', title: 'New section', body: 'Add your text
    here.' }; },
  render: function (block, ctx) {
    var sec = ctx.el('section', { 'class': 'engine-section' });
    sec.appendChild(ctx.el('h2', {
      'class': 'engine-section-title', text: block.title || '',
      'data-edit-file': ctx.file, 'data-edit-text': ctx.path + '.title'
    }));
    sec.appendChild(ctx.el('div', {
      'class': 'engine-section-body', html: block.body || '',
      'data-edit-file': ctx.file, 'data-edit-html': ctx.path + '.body'
    }));
    return sec;
  }
}
```

What makes the title and body *editable* is nothing more than the data-edit-file + data-edit-text/data-edit pairs, with the path built off ctx.path. The part author writes no editor code.

A list-bearing part — gallery — adds a lists{} schema and stamps data-item-index on each child:

```
gallery: {
  label: 'Photo gallery', category: 'Media', icon: '[#]',
  blank: function () { return { type: 'gallery', title: 'New gallery', images: [] }; },
  lists: { images: { label: 'Image', fields: [
    { key: 'src', type: 'image', label: 'Image' },
    { key: 'alt', type: 'text',  label: 'Alt text' }
  ] } },
  render: function (block, ctx) {
    var sec  = ctx.el('section', { 'class': 'engine-section' });
    var grid = ctx.el('div', { 'class': 'engine-gallery',
      'data-edit-file': ctx.file,
      'data-edit-list': ctx.path + '.images',
      'data-edit-list-label': 'images' });
```

```
    (Array.isArray(block.images) ? block.images : []).forEach(function (im, j) {
      var src = (im && im.src) ? im.src : '';
      grid.appendChild(ctx.el('img', {
        src: ctx.mediaUrl(src), alt: (im && im.alt) || '', loading: 'lazy',
        'data-item-index': j,                          // TRUE array index
        'data-edit-file': ctx.file,
        'data-edit-image': ctx.path + '.images[' + j + '].src'
      }));
    });
    sec.appendChild(grid); return sec;
  }
}
```

The container carries `data-edit-list` (so the editor's list machinery binds to it); each child carries an editable path and its true `data-item-index`. The `Array.isArray(...)` guard means a malformed `images` degrades to an empty grid instead of throwing.

5.4 How the renderer drives a part

`renderOne(block, ctx)` is the only thing that calls your part, in a fixed sequence worth knowing because it tells you what is guaranteed when your code runs:

1. `ensureCSS()` — inject the catalogue stylesheet once (lazy, idempotent);
2. refresh `STATPAL` from the live `--cat-*` palette (theming current);
3. look up `BLOCKS[block.type]`, falling back to `text` for unknown types;
4. call your `render(block, ctx)` to get a node;
5. stamp engine bookkeeping: `data-engine-block`, `data-engine-index`, `data-engine-path`;
6. apply block `style` and per-text `textStyles` via `EngineStyle`;
7. call your `init(node, block, ctx)` inside a `try/catch` — a throw is non-fatal.

5.5 Worked example: adding a part to `site-blocks.js`

`site-blocks.js` ships empty, with a commented template. Here is the full procedure to add a bespoke `hero`. Everything goes inside the file's IIFE.

```
(function () {
  'use strict';
  if (!window.CatalogBlocks) return;   // engine not loaded -> no-op, never throw

  window.CatalogBlocks['hero'] = {
    label: 'Hero', category: 'Layout', icon: '*',          // 2. picker metadata
    blank: function () {                                    // 3. default block; MUST set
     type
      return { type: 'hero', title: 'Big headline', subtitle: 'Supporting line' };
    },
    fields: [                                               // 4. inspector inputs
      { key: 'title',    type: 'text', label: 'Headline' },
      { key: 'subtitle', type: 'text', label: 'Subtitle' }
    ],
    render: function (block, ctx) {                         // 5. build DOM + data-edit-*
```

```
      ensureHeroCSS();
      var sec = ctx.el('section', { 'class': 'engine-section catalog-hero' });
      sec.appendChild(ctx.el('h1', { 'class': 'catalog-hero-title',
        text: block.title || '',
        'data-edit-file': ctx.file, 'data-edit-text': ctx.path + '.title' }));
      sec.appendChild(ctx.el('p', { 'class': 'catalog-hero-sub',
        text: block.subtitle || '',
        'data-edit-file': ctx.file, 'data-edit-text': ctx.path + '.subtitle' }));
      return sec;
    }
  };

  function ensureHeroCSS() {                              // 6. inject CSS once,
    namespaced, themed
    if (document.getElementById('catalog-hero-styles')) return;
    var s = document.createElement('style');
    s.id = 'catalog-hero-styles';
    s.textContent = [
      '.catalog-hero{text-align:center;padding:3rem 1rem}',
      '.catalog-hero-title{font-size:clamp(2rem,6vw,4rem);margin:0;color:var(--site-ink,#
    fff)}',
      '.catalog-hero-sub{opacity:.8;margin:.6rem 0 0;color:var(--site-ink,#fff)}'
    ].join('\n');
    document.head.appendChild(s);
  }
})();
```

Step 7 — that's it. Reload a draft page. The `hero` appears in the picker, can be added, its text is click-to-edit on the page *and* editable from the inspector, its styling is namespaced and theme-aware, and Save persists it like any other block. You wrote one object and one CSS helper — no changes to `sections.js`, `page.js`, the editor, or the backend. If your part needs runtime behaviour, add `init`; if it has an array, add a `lists{}` schema and stamp `data-item-index` per child, exactly like `gallery`.

6 The rules every part must follow

A part is portable and composable only if it obeys a handful of rules. Treat this as a checklist. Each rule exists because violating it breaks composition with *other* parts, or portability across *other* sites.

☐ **Namespace every CSS class under `catalog-`.** Class names like `.catalog-hero-title`, not `.title`. *(Two parts on one page, or one part on a foreign site, must not leak styles into each other.)*

☐ **Inject CSS once, lazily, on first render.** Guard with a unique `id` and append a single `<style>`. *(A part must look right on any page, and must not re-inject for each instance.)*

☐ **Emit `data-edit-*` for everything editable.** Text → `data-edit-text`; rich text → `data-edit-html`; images → `data-edit-image`; arrays → `data-edit-list`; an attribute → `data-edit-attr`; always pair with `data-edit-file`. *(The only way the generic editor can edit your part.)*

☐ **Build paths off `ctx.path`.** Never hard-code `sections[2]`; use `ctx.path + '.title'`. *(The same part renders at different indices and inside groups at deep paths like `sections[1].children[0]`.)*

☐ **Stamp `data-item-index` on every list child** — its *true* array index. *(On-page drag reorders the right item even when some items are filtered out of the render.)*

☐ **Guard with `Array.isArray`/falsy checks — degrade, never throw.** *(Content can be partial mid-edit; a part that throws takes the whole render down, a part that degrades shows an empty-but-valid section.)*

☐ **Be theme-aware via CSS variables** — `var(--site-accent, ...)`, `var(--cat-1, ...)`, always with a fallback. *(A theme change in JSON must re-colour your part with no code edit; the fallback keeps it good with no theme at all.)*

☐ **No site-specific strings in `blocks.js`** — brands, real pages, real URLs belong in `site-blocks.js` or content. *(`blocks.js` is engine; it must drop into any site unchanged.)*

> *The unifying idea.* Every rule is a corollary of one principle: a part is a black box that talks to the engine only through the documented contract (the registry keys, `ctx`, `data-edit-*`, the CSS variables) and otherwise keeps entirely to itself. Stay inside that boundary and your part composes with every other part and travels to every other site.

7 The editing protocol

You make a part editable purely by tagging its DOM. The editor (`edit-mode.js`, documented fully in the separate *Editor — Developer Guide*) scans for these attributes and wires the interactions. As a part author you need only the producer side:

Attribute (with `data-edit-file`)	Makes editable	Example value
`data-edit-text`	plain text of the node	`sections[2].title`
`data-edit-html`	rich text (innerHTML)	`sections[2].body`
`data-edit-image`	an image (opens the picker)	`sections[2].images[0].src`
`data-edit-attr`	one attribute, `path|attr`	`sections[2].url|href`
`data-edit-bg`	a background image	`sections[2].style.bgImage`
`data-edit-list`	a reorderable array	`sections[2].images`
`data-item-index`	(on each list child) its index	`0`, `1`, `2`

Every value is a **data-path**: a dotted/bracketed address into the page JSON, such as `sections[2].items[0].va` The editor resolves the path, edits that spot in the in-memory data, re-renders, and on Save writes the JSON back. You never write that resolution code — you only emit the right path, and you always build it from `ctx.path` so it is correct at any depth. The deep editor internals are covered in the *Editor — Developer Guide*; cross-refer it when you need the consumer side.

8 The two namespaces: `engine-` vs `catalog-`

Pagesmith uses exactly two prefixes, and the split is meaningful:

Prefix	Owns
`engine-`	the skeleton, editor, and page structure: `.engine-section`, `#engine-sections`, `window.EngineStyle/EngineTheme/EngineEdit`, the `engine-data-ready/engine-rebind` events, `data-engine-path`
`catalog-`	the parts: `.catalog-carousel`, `.catalog-statcard`, `.catalog-hero`, `window.Catalog`, `window.CatalogBlocks`

The reason for two namespaces is the engine-versus-instance / skeleton-versus-parts boundary *made visible*. Anything `engine-` is structural plumbing owned by the skeleton and editor: stable, rarely touched. Anything `catalog-` is a part's own surface: parts may be added, removed, or replaced freely. Reading a stylesheet or a stack trace, the prefix instantly tells you which layer you are in and which rules apply — `engine-` classes are shared and must not be reused by parts; `catalog-` classes belong to one part and must not collide with another.

9 Theming

Colour and brand are kept *out of the code* and expressed as CSS custom properties. The engine and every part reference variables like `var(--site-accent)`; `theme.js` is the one place that sets them, from data. It reads a `theme` object — from `global.json` (site default) and/or a page's own JSON (override) — and writes the matching variables onto `<html>`. It is event-driven: `global.json`'s theme sets the defaults; a page's theme arrives after and overrides.

`theme` key	CSS variable	Role
`accent`	`--site-accent`	links, buttons, chart bars
`text`	`--site-ink`	main foreground / ink
`bg`	`--site-bg`	page background colour
`palette:[..]`	`--cat-1` … `--cat-N`	data-viz palette
`spotlight`	`--cat-spot`	the spotlight gradient
`bgImage`	`--site-bg-image`	full-bleed background photo

Two details worth internalising. **Clearing reverts live:** when a `theme` key is absent, `theme.js` actively `removeProperty()`s the variable rather than skipping it, so deleting a colour in the editor immediately falls back to the stylesheet default. **`--cat-*` is overridable but optional:** parts read the palette via `STATPAL = palette()`, which pulls `--cat-1..8` from the live computed style and falls back to a built-in neon default, so the catalogue looks good with no theme *and* a site can recolour all data-viz with one `theme.palette` array.

The practical upshot for a part author: **never write a hex colour without a variable in front of it** — `var(--site-accent, #4f7cff)`, variable first, fallback second. That single habit makes a part both themeable and safe with no theme.

10 The backend in one section

The backend is a small set of Azure Functions under `api/`. Its defining property mirrors the front end: **it is generic by *shape*, so it needs zero site configuration.** Nothing names your site, pages, or folders. The genericity comes from two allow-lists that match *patterns*, not names:

- **Content files** (`_lib/content.js`): a content file is any flat lowercase JSON name matching `^[a-z0-9][a-z0-9-]{0,39}\.json$`. That covers `global.json` and every `page-<slug>.json`, with no per-site list and no path traversal.
- **Media paths** (`_lib/media.js`): allow-listed by shape — a root file (`logo.png`) or one folder deep (`<folder>/<file>`), safe charsets, no `..`, no leading slash.

The endpoints, all gated on the SWA `editor` role server-side (`_lib/auth.js`, checking the `x-ms-client-principal` header — defence in depth on top of route-level role config):

Endpoint	What it does
`POST /api/save-content/{file}`	validate the filename against the content shape, write that one JSON file to the *draft* container (the editor's Save)
`POST /api/promote`	*publish*: copy every content file draft→live (skipping `visibility:'dev'` pages), then sync media draft→live
`POST /api/revert`	the inverse: copy content live→draft, discarding unpublished edits
`POST /api/media-upload`	accept a base64 file, validate path/extension, write to draft media; re-encode rasters to WebP + `-mobile`/`-thumb` variants (unless full-quality)
`POST /api/submit-questionnaire`	append a visitor footer-form submission to storage

The "draft vs published" model is just two sets of blob containers. You edit against draft; Publish copies draft→live; the live site reads live containers. The hostname switch deciding which set you see lives entirely in `site-config.js` (`prodHosts`) on the front end. There is genuinely no site-specific configuration in the API: a new site reuses these Functions verbatim and only points new storage containers at them via environment settings.

11 Standing up a new site

Because the engine is generic, a new site is *new values + new content*, not new code. The full checklist:

1. **Copy the project.** You keep the entire engine (`app/javascript/*` except `site-config.js`/`site-blocks.js`, `api/`, `css/layout.css`, the shells). Everything you change is instance.
2. **Rename the site** — edit `app/javascript/site-config.js`: `siteName` (tab-title suffix); `branding.wordmark` (editor toolbar); `pages` (friendly names + optional background slots; *not* required for a page to exist); `mediaTabs` (picker folder tabs); `socialIcons` (map platform → SVG `<symbol>` id + label + tracking key); `prodHosts` (**leave empty while building** — every host is then a draft; add your real domain only when going live); `fullQualityMedia` (media prefixes that skip WebP/transcode — mirror in the API's `MEDIA_FULL_QUALITY_PREFIXES`).
3. **Replace the content** — edit `app/data/`: `global.json` (nav, social, footer, questionnaire, site-wide `theme`) and the `page-*.json` files — or just build pages in the editor and Save.
4. **Restyle.** Most colour flows from `theme.accent` (and the rest of `theme`) in `global.json` via the `--site-*` variables. Tweak `header.css`/`footer.css` for chrome; `layout.css` is generic.
5. **Add custom parts (optional)** — `app/javascript/site-blocks.js`, following Section 5 and the rules in Section 6. The generic catalogue already gives you text, headings, galleries, video, carousels, embeds, links, quotes, buttons, charts, stat-cards, spotlights, rankings, and groups, so most sites need *no* custom parts.
6. **New pages need no code.** Any `/p/<slug>` URL renders `data/page-<slug>.json` through the universal shell. For a clean top-level URL, add one rewrite in `staticwebapp.config.json` (`/showcase` is the worked example).
7. **Deploy & go live.** Deploy the `./app` Static Web App linked to `./api` Functions. Edit on a draft host; **Publish** to copy draft containers to live; finally set `prodHosts` to your real domain so the live host turns the editor off and tracking on.

You never edit the engine. That is the promise the whole architecture exists to keep: a constant, battle-tested core, and a thin shell of values and content that is all that ever differs between one site and the next.